LAM Implementation Specification

Oct 18, 2026 · @Mr.Forever

Project: Learning Adaptation Model (LAM) · **Type:** Formal implementation specification · **Version:** 1.0, draft for engineering review · **Status:** Proposed; approval required before implementation of complex learning logic · **Applies to:** the standalone LAM repository and its integrations with Qrie and other learning applications.

Primary references: LAM Complete Product and Engineering Requirements; LAM Engineering Handbook; LAM Engineering Standards and Development Lifecycle; LAM Product Requirements Document (both existing versions).

1. Purpose

This document translates LAM's existing product requirements and engineering standards into a concrete implementation contract. It defines the system boundaries, module responsibilities, domain entities, data contracts, processing lifecycle, algorithm interfaces, failure behavior, testing requirements and release gates required to build LAM incrementally.

The purpose is to ensure that multiple engineers can implement different parts of LAM without inventing incompatible assumptions about learner state, mastery, diagnosis, recommendation selection, event processing or persistence.

This document does not claim that any capability described here is already implemented.

1.1 Implementation objectives

The implementation must:

1. Accept authenticated, validated learning evidence from Qrie and other authorized clients.
2. Preserve a durable, auditable history of learning events.
3. Maintain persistent learner state for individual concepts.
4. Distinguish independent performance from assisted performance.
5. Estimate mastery without claiming more certainty than the evidence supports.
6. Represent possible misconceptions as hypotheses rather than unquestioned facts.
7. Recommend appropriate next actions while respecting prerequisite constraints.

8. Schedule future retrieval and retention assessments.
9. Expose stable, versioned APIs and integration contracts.
10. Support deterministic testing and reproducible decisions.
11. Protect student information through tenant isolation, access control, data minimization and defined retention policies.
12. Provide evidence that the system improves independent mastery, retention and transfer.
13. Keep shared-model training outside the live event-processing path.
14. Permit future algorithm and model improvements without requiring changes to Qrie's internal implementation.

1.2 Governing principle

The implementation follows: **Observe, Diagnose, Teach, Test, Update Learner Model, Select Next Best Action, Revisit Later.**

LAM decides how evidence changes learner state and what learning action should happen next. The host application delivers the selected learning experience and returns the resulting evidence.

2. Authority and document precedence

The existing LAM PRDs define product intent. The engineering handbook and engineering standards define mandatory engineering behavior. This specification defines the implementation contract derived from those sources.

If requirements conflict, engineers must not silently choose whichever version is easier to implement. The conflict must be entered in the decision register in section 17, and the responsible technical owner must approve the resolution before the affected behavior is implemented.

Security, privacy, authentication, tenant isolation, auditability, and the prohibition against treating assisted success as independent mastery cannot be bypassed for convenience.

A requirement is not complete merely because an endpoint responds successfully. Completion requires implementation, automated tests, manual verification where specified, and documented evidence.

3. Scope and system boundaries

3.1 LAM owns

- Learning-event ingestion and validation
- Evidence provenance and the event ledger
- Persistent learner state
- Mastery estimation and uncertainty
- Independent and assisted performance tracking
- Misconception hypotheses and diagnostic evidence
- Assistance policy and productive-struggle decisions
- Prerequisite-aware sequencing
- Next-best-action selection
- Retrieval and forgetting management
- Teaching-strategy effectiveness evidence
- Session-level learning analysis
- Versioned APIs and SDK contracts
- Shared-model governance and lifecycle
- Evaluation, observability and decision auditability

3.2 Qrie and other host applications own

- Student authentication at the application level and delivery of trusted identity to LAM
- Student-facing interfaces
- Curriculum content and lesson generation
- Question, hint, explanation and worked-example content
- Assessment delivery and collection of responses
- Answer verification and grading provenance
- Audio, voice, animation and interactive classroom rendering
- School, teacher and student administration
- Presenting LAM recommendations to learners
- Reporting the actual intervention delivered and its outcome

LAM may request a diagnostic question, a hint, independent practice, prerequisite remediation or a review. It does not generate the corresponding lesson or classroom content.

3.3 Architectural boundary

LAM must be independently deployable and consumable through a stable REST API and SDKs. Qrie must not import LAM's internal Python modules or depend on its database schema; integration occurs only through documented external contracts. LAM must not depend on Qrie's internal application packages.

3.4 Initial deployment topology

Proposed default: start with a modular monolith. Use one independently deployable LAM service with well-separated modules. Keep shared-model training and evaluation in a separate offline execution path. Do not introduce a separate network service for every subsystem unless operational evidence justifies it.

4. Required technology and repository structure

The implementation uses the technology choices established by the engineering documents:

- Python for domain and application logic
- FastAPI for HTTP endpoints
- Pydantic v2 for boundary validation
- SQLAlchemy 2.0 with asynchronous database access, and asyncpg for PostgreSQL connectivity
- Alembic for database migrations
- PostgreSQL for durable event and learner-state storage
- pytest for automated tests
- Ruff for linting and formatting, and mypy in strict mode for static type checking
- OpenAPI for the HTTP contract
- A documented Python SDK and a TypeScript client or adapter for external consumers

Exact dependency versions must be pinned in the project configuration and updated through reviewed changes.

4.1 Suggested repository structure

```
lam/
├── pyproject.toml
├── README.md
├── CONTRIBUTING.md
├── .env.example
├── alembic.ini
├── alembic/
│   ├── env.py
│   └── versions/
├── docs/
│   ├── LAM_IMPLEMENTATION_SPECIFICATION.md
│   ├── architecture.md
│   ├── api-contracts.md
│   ├── domain-model.md
│   ├── algorithm-specifications.md
│   ├── security-and-privacy.md
│   ├── testing-and-verification.md
│   ├── deployment-and-operations.md
│   └── decisions/
├── src/
│   └── lam/
│       ├── api/
│       │   ├── routes/
│       │   ├── dependencies.py
│       │   ├── error_handlers.py
│       │   └── schemas/
│       ├── application/
│       │   ├── services/
│       │   ├── commands/
│       │   └── queries/
│       ├── domain/
│       │   ├── events/
│       │   ├── learner_state/
│       │   ├── mastery/
│       │   ├── diagnosis/
│       │   ├── decisions/
│       │   ├── retrieval/
│       │   ├── curriculum/
│       │   └── shared/
│       ├── infrastructure/
│       │   ├── database/
```

```
|       |   ├── repositories/
|       |   ├── authentication/
|       |   ├── observability/
|       |   └── integrations/
|       └── main.py
├── sdk/
|   ├── python/
|   └── typescript/
├── tests/
|   ├── unit/
|   ├── integration/
|   ├── contract/
|   ├── security/
|   ├── end_to_end/
|   └── fixtures/
└── evaluation/
    ├── scenarios/
    ├── datasets/
    ├── baselines/
    └── reports/
```

This structure is a proposed implementation layout, not a claim about the current repository. It can be adjusted if the existing repository already has an equivalent structure.

4.2 Dependency direction

The permitted dependency direction is **API, then application services, then domain logic**. Infrastructure implements the persistence and external-service interfaces that application services consume.

Domain code must not directly depend on FastAPI, HTTP request objects, ORM sessions, external model providers or database implementation details. Business logic must not be hidden inside route handlers, ORM hooks, SDK wrappers or background-job callbacks.

5. Core domain model

The domain model must distinguish recorded observations from derived conclusions.

5.1 Learning event

A learning event represents an observable learning interaction, such as an attempt, hint, explanation, assessment or retrieval review. Required or conditionally required fields are validated against a versioned schema.

Field	Meaning
`event_id`	Unique event identifier used for idempotent ingestion
`tenant_id`	Organization boundary, derived or verified server-side
`student_id`	Learner identity resolved and authorized by the server
`session_id`	Learning session reference
`concept_ids`	Concepts addressed by the activity
`item_id`	Question or assessment reference
`timestamp`	Event time, timezone-aware and normalized for persistence
`event_type`	Attempt, hint, explanation, assessment, review, or another defined type
`correctness`	Outcome with verification provenance, or unknown
`attempt_number`	Position in the attempt sequence, when applicable
`assistance_level`	Amount of assistance received, when applicable
`hints_used`	Hint metadata, when available
`response_time_ms`	Response duration, when available and meaningful
`self_reported_confidence`	Learner-stated confidence, when available
`strategy_id`	Teaching strategy identifier, when applicable
`intervention_id`	Intervention identifier, when applicable
`difficulty`	Difficulty value or explicitly unknown

Field	Meaning
`assessment_mode`	Independent, assisted, delayed, transfer, or another defined mode
`provenance`	Origin and verification status of the evidence
`schema_version`	Version of the event contract

Additional operational fields may include a server-generated ingestion timestamp, request identifier, policy version and audit metadata.

A client-supplied `student_id` or `tenant_id` must never independently authorize access to that student's records. Missing optional values must remain unknown; they must not silently become `0`, `false` or an arbitrary default.

5.2 Learner state

Learner state is the current evidence-supported estimate of what a student understands about a concept. It is keyed by the authorized learner and concept, with tenant, subject, curriculum and prerequisite context where required.

It supports:

- Overall mastery estimate and uncertainty
- Independent mastery estimate and uncertainty
- Assisted performance
- Evidence counts and evidence references
- Misconception hypotheses
- Retention and retrieval history
- Strategy and intervention outcomes
- Last practice time and next review time
- State version
- Algorithm, policy, graph and schema versions used to produce it
- A reason explaining why the state changed

Historical state snapshots, or an equivalent reconstructable history, must be available for audit and debugging.

5.3 Misconception hypothesis

A misconception hypothesis is a possible explanation for a student's observed difficulty. It contains:

- Hypothesis identifier and student and concept references
- Hypothesis category
- Supporting evidence references, and contradictory evidence references when available
- Confidence estimate and its meaning
- Lifecycle status
- Creation and update timestamps
- Diagnosis-policy version
- Reason for confirmation, revision or retirement

Possible categories include conceptual misunderstanding, missing prerequisite, procedural error, arithmetic error, misreading, guessing, retrieval failure, carelessness and flawed reasoning. These are candidate explanations, not permanent student labels.

5.4 Recommendation

A recommendation is a learning action that the host application should deliver. It includes:

- Recommendation identifier, action type, target concept and suggested strategy
- Assistance level
- Relevant evidence references and reason codes
- A brief user-safe rationale
- An uncertainty or insufficient-evidence indicator
- Required prerequisite conditions
- Learner-state version, policy version and curriculum-graph version
- Model version, when a model is involved
- A safe fallback or explicit inability to recommend

A recommendation must not contain internal chain-of-thought or unsupported claims about a student's ability.

5.5 Retrieval plan

A retrieval plan represents when and how a concept should be revisited. It includes:

- Learner and concept identifiers
- Last qualifying practice event and last independent success
- Last retrieval outcome
- Current review interval and next scheduled review
- Scheduling-policy version
- Reason for scheduling or rescheduling
- Whether the review is overdue
- Whether the review is intended to verify durable mastery

The exact scheduling algorithm is subject to section 12 and the decision register.

6. API contracts and request lifecycle

All external APIs must be versioned and documented in OpenAPI.

6.1 Required initial API surface

This is the minimum proposed v1 surface. Exact endpoint paths and response schemas must be finalized in the API contract before implementation.

Endpoint	Purpose
`POST /v1/events`	Validate and ingest learning evidence
`GET /v1/learners/{learner_id}/state`	Retrieve authorized learner state
`GET /v1/learners/{learner_id}/concepts/{concept_id}/state`	Retrieve state for one concept
`POST /v1/recommendations`	Request a next-best-action recommendation
`GET /v1/learners/{learner_id}/reviews`	Retrieve scheduled or overdue reviews
`GET /health/live`	Check process liveness
`GET /health/ready`	Check whether required dependencies are available

The API may use a server-resolved learner context rather than exposing every identifier in the path. The final contract must not rely on client-provided IDs as proof of authorization. Endpoints for diagnosis review, session closure, administrative access or correction workflows are added only when their contracts and permissions are defined.

6.2 Event-ingestion lifecycle

1. Assign or validate a request identifier.
2. Authenticate the caller.
3. Resolve tenant and learner permissions.
4. Validate the event against its schema version.
5. Validate identifiers, value ranges, event types, timestamps and provenance.
6. Check event identity and idempotency.
7. Persist valid evidence durably.
8. Apply eligible deterministic state changes according to the selected milestone's policy.
9. Record the state version and processing outcome.
10. Return a stable response describing the result.

If processing fails, the system must not silently lose the event or partially apply a state update. Where event ingestion and state update are required to be atomic, they must share a transaction or use a documented recoverable processing design with equivalent guarantees.

6.3 Recommendation lifecycle

1. Authenticate and authorize the caller.
2. Resolve the correct learner and tenant.
3. Load the relevant learner state.
4. Load or validate the curriculum-graph version and prerequisite information.
5. Determine whether evidence is sufficient for a specific recommendation.
6. Apply mandatory safety, prerequisite and progression constraints.
7. Evaluate eligible next actions under the versioned decision policy.
8. Produce the recommendation with reason codes, relevant versions and uncertainty.
9. Persist a decision audit record when required by the selected policy.
10. Return the recommendation to the host application.

If the learner state or curriculum information is unavailable, the service returns a documented safe fallback or explicit error. It must not invent confident mastery estimates or recommend actions that violate known prerequisites.

6.4 Error contract

Errors use a stable machine-readable format. A proposed shape:

```
{
  "error": {
    "code": "INVALID_EVENT_SCHEMA",
    "message": "The submitted event does not satisfy the required schema.",
    "request_id": "server-generated-request-id",
    "details": []
  }
}
```

The final implementation defines stable error codes for at least:

- Invalid request or schema
- Missing or invalid credentials
- Unauthorized tenant or learner access
- Unknown resource
- Duplicate event
- State-version conflict
- Invalid curriculum or concept reference
- Rate limiting
- Dependency unavailable
- Internal processing failure

The exact HTTP status and duplicate-event behavior must be specified consistently. Errors must not expose stack traces, secrets, raw student data or internal database details.

7. Authentication, tenant isolation and event integrity

7.1 Authorization requirements

Every learner-data operation verifies permissions server-side. The system enforces:

- Tenant-scoped access and learner-level access
- Appropriate service-to-service permissions
- Explicit authorization for administrative or cross-learner operations
- Authorization for background jobs and asynchronous workers
- Separation between development, test, staging and production credentials

A student ID in a URL or request body is not authorization.

7.2 Idempotency

Repeated delivery of the same valid event must not apply the same logical learner-state update twice. The database enforces the necessary uniqueness constraints; application-level memory checks are insufficient.

The system distinguishes between a new event, a repeated event with equivalent content, and an attempted reuse of the same event identifier with conflicting content. The last case is rejected or quarantined under a documented policy and recorded for audit.

Acceptance criterion: submitting an identical event repeatedly produces one logical state transition, with a documented response for duplicate submissions.

7.3 Concurrent updates

Concurrent events must not silently overwrite each other. Use transactions and optimistic locking or an equivalent concurrency-safe mechanism, and verify the expected state version before committing. On conflict, follow a defined retry or conflict-response policy. Never blindly repeat non-idempotent operations.

7.4 Corrections

Historical evidence must not be silently rewritten or deleted to hide an earlier observation. A correction workflow records:

- The original event reference
- The correction or superseding event
- The reason and provenance
- The actor or authorized process responsible
- The state-recalculation behavior
- The affected state versions

The precise correction and privacy-deletion semantics require an approved policy.

8. Learner-state engine and mastery estimation

The learner-state engine converts validated evidence into a persistent, versioned estimate of understanding.

8.1 State-update contract

Each update receives: the prior state or an explicit initial-state representation; a validated event; the applicable curriculum context; the selected learner-model algorithm and version; relevant policy configuration; and any required state-version precondition.

Each update returns: the updated state if a valid update can be computed; a state version; evidence references used; a reason code; the algorithm and policy versions; and an explicit outcome when the evidence is insufficient.

Domain logic is deterministic wherever the underlying policy is deterministic.

8.2 Independent and assisted performance

Independent and assisted evidence are represented separately. A correct answer after a hint, worked example, partial solution or direct explanation cannot automatically establish independent mastery. The implementation records enough assistance context to determine how the answer was obtained. If assistance metadata is missing, the system must not assume the response was independent.

8.3 Bayesian Knowledge Tracing

The PRD identifies Bayesian Knowledge Tracing (BKT) as the baseline mastery-estimation approach for M1. Before implementation, `docs/algorithm-specifications.md` must define:

- The exact BKT variant
- Initial knowledge probability
- Learning, guessing and slipping parameters, with their provenance and configuration
- How evidence is classified as independent or assisted
- How invalid, ambiguous or unverified outcomes are handled
- How uncertainty is represented and exposed
- How state is initialized when no evidence exists

- How historical events can be replayed consistently
- How parameter changes are evaluated and versioned

Do not claim that a BKT probability is a calibrated probability of real-world mastery until calibration has been evaluated. Do not convert missing evidence into negative evidence.

8.4 State initialization

A new learner-concept pair begins in a documented insufficient-evidence state. The system distinguishes an unobserved concept from one that has been observed and repeatedly answered incorrectly. The exact initial numeric estimates and uncertainty representation require approval, and must not be selected merely to make a dashboard look complete.

8.5 Mastery verification

Mastery confirmation relies on independent assessment evidence. The implementation separates:

- Provisional evidence of progress
- Assisted success
- Independent success
- Delayed retention success
- Transfer performance
- Confirmed mastery under the approved policy

A single correct answer, lesson completion, high engagement or assisted success must not independently confirm durable mastery. The existing documents describe both mastery-estimate thresholds and evidence-count requirements. One authoritative policy must be approved before the relevant transition is implemented (decision D-01).

9. Misconception diagnosis

9.1 Diagnostic objective

The diagnosis engine evaluates plausible explanations for a learner's observed difficulty. It does not automatically label every incorrect answer as a misconception.

9.2 Required behavior

The engine must:

1. Examine relevant validated evidence.
2. Identify plausible candidate explanations.
3. Preserve uncertainty where the evidence is ambiguous.
4. Distinguish a prerequisite gap from a conceptual misunderstanding where evidence permits.
5. Consider procedural errors, arithmetic errors, misreading, retrieval failure, guessing and slips.
6. Request additional diagnostic evidence when necessary.
7. Record supporting and contradictory evidence.
8. Update the lifecycle of a hypothesis when new evidence arrives.
9. Retain an audit trail for changes in diagnostic status.

9.3 Hypothesis lifecycle

The proposed lifecycle is: **open, then confirmed, then revised or retired.**

The PRD and broader requirements describe a possible confirmation rule of confidence at least 0.8 with at least two independent pieces of evidence.

Decision required (D-04): approve the confidence definition, evidence-independence criteria and confirmation rule before implementing automatic confirmation. Two records must not automatically count as independent evidence if they originate from the same underlying attempt or a duplicated event.

A hypothesis must be revised or retired when sufficient contradictory evidence, or later independent mastery, supports that decision.

9.4 Diagnostic outputs

A diagnostic response includes candidate categories, evidence references, confidence or uncertainty, whether confirmation is required, the next diagnostic action if applicable, and the diagnosis-policy version. A low-confidence diagnosis must not be presented to the learner or teacher as a verified fact.

10. Adaptive decision engine and assistance policy

10.1 Responsibility

The decision engine selects the next appropriate learning action from the available eligible actions. It does not generate the content of the lesson, hint, explanation or question; the host application delivers the selected intervention.

10.2 Candidate actions

The initial policy supports the action categories described by the PRD:

- Independent practice
- Metacognitive or rethink prompt
- Conceptual question
- Hint and stronger hint
- Analogy or alternative explanation
- Similar worked example
- Partial solution
- Direct explanation
- Prerequisite remediation
- Diagnostic question
- Delayed retrieval review
- Independent mastery verification

The final action enum and its mapping to host-application capabilities are defined in the API contract.

10.3 Assistance ladder

The proposed levels run from an independent attempt through progressively stronger help to a direct answer.

Level	Meaning
0	Independent attempt
1	Rethink prompt

Level	Meaning
2	Conceptual question
3	Small hint
4	Stronger hint
5	Similar worked example
6	Partial solution
7	Full explanation
8	Direct answer

These levels are internal policy values. The UI must not expose the numeric level as if it were a student rating. The specification defines valid transitions, escalation limits, reset conditions, and when a fresh independent attempt is required. The ladder must not force repeated hints when a student needs a different explanation or prerequisite remediation (decision D-05).

10.4 Decision priority

The requirements describe this general priority ordering:

1. Safety and stop conditions
2. Blocking prerequisite gaps
3. Confirmed misconceptions
4. Overdue retrieval
5. Mastery verification
6. Curriculum and learning goals
7. Difficulty fit

This ordering must be made executable. The implementation defines how ties are resolved, what happens when multiple conditions apply, and which conditions are hard constraints rather than scoring preferences (decision D-06).

10.5 Recommendation rationale

Each recommendation includes a short, user-safe rationale grounded in evidence and policy. Examples:

- "Review the prerequisite concept before continuing."
- "Try one independent question to verify your understanding."
- "This concept is due for retrieval practice."
- "More evidence is needed before selecting a specific intervention."

The system must not expose internal chain-of-thought or state a diagnosis with more certainty than the evidence supports.

10.6 Fallback behavior

If the service lacks sufficient evidence to choose a specific intervention, it returns a documented safe fallback or an explicit insufficient-evidence result. A fallback must not silently bypass prerequisite constraints or mark mastery as established.

11. Curriculum graph integration

LAM consumes a versioned curriculum/concept graph through a stable adapter or contract. It must not maintain an independently drifting copy of the host application's curriculum.

11.1 Required graph information

The graph contract provides concept identifiers, subject and grade context where relevant, prerequisite relationships, the graph version, concept availability and status, and any required curriculum-goal metadata. The exact graph schema must be documented before M2 integration (decision D-10).

11.2 Validation

The adapter validates that referenced concepts exist in the supplied graph version and that the graph satisfies its structural rules. At minimum, prerequisite relationships are checked for cycles where the intended graph model requires a directed acyclic graph.

11.3 Decision behavior

The decision engine must not recommend progressing past a hard prerequisite when the approved policy says that prerequisite blocks progression. If the graph is missing, invalid, stale or incompatible, the service uses an explicitly defined fallback or returns an error. It must not invent prerequisite relationships.

12. Retrieval and forgetting management

12.1 Purpose

The retrieval scheduler determines when a learner should revisit a concept and what evidence should be collected during that review. It considers the learner's history, independent success, assisted performance, failure, elapsed time, retention evidence and the scheduling-policy version.

12.2 Required scheduling inputs

Where available, the scheduler uses:

- Last qualifying practice timestamp and last independent success
- Previous review interval and previous retrieval outcome
- Evidence quality and verification provenance
- Current mastery estimate and uncertainty
- Open prerequisite or misconception conditions
- Current time and the applicable scheduling policy

Missing inputs remain unknown and must not be treated as evidence that the learner has forgotten a concept.

12.3 Proposed baseline behavior

The existing requirements suggest a baseline review interval of one day, with changes depending on the outcome: expand the interval after independent success, retain or reduce it after assisted success, and reset it after failure. These behaviors must be converted into an exact, versioned algorithm with defined units, boundaries and time-zone handling before implementation.

Decision required (D-07): approve the exact interval multiplier, minimum and maximum intervals, treatment of overdue reviews, behavior after repeated failures, and the interaction between review scheduling and mastery verification.

The scheduler must not treat the passage of time, or completion of a scheduled review, as proof of retained mastery.

12.4 Delayed retrieval

A delayed assessment must be identifiable as delayed retrieval evidence, and its result linked to the concept and the relevant review schedule. The system distinguishes a scheduled review from a successfully completed and independently verified review.

13. Event ledger, persistence and transaction requirements

13.1 Persistence responsibilities

PostgreSQL stores the durable records required for learning events, learner-concept state, state versions or snapshots, misconception hypotheses, retrieval schedules and outcomes, decision audit records, policy and schema version references, and required processing or recovery metadata. The final relational schema is documented in `docs/domain-model.md` and implemented through Alembic migrations.

13.2 Database invariants

The database and application enforce:

1. Event identity and idempotency.
2. Tenant-scoped learner-data access.
3. Referential integrity where applicable.
4. Concurrency-safe learner-state versions.
5. Durable event history.
6. Valid state and event schema versions.
7. Transactional consistency for operations required to be atomic.
8. No silent loss of evidence following recoverable failures.

13.3 Transactional behavior

For a state-changing event, the system defines which operations commit together. If the event insert and learner-state update must be atomic, a partial failure rolls back both. If the design uses asynchronous processing, it provides durable pending work, retry behavior, deduplication and observable processing status.

The system must not return a successful final-processing result when the event has only been accepted but not yet processed, unless the response clearly distinguishes those states.

13.4 Recovery and replay

The implementation documents how failed processing is recovered. Where replay is supported, the result must be reproducible from the normalized event history, initial state, and the applicable algorithm and policy versions. Historical evidence must not be reprocessed under a new policy without recording which version produced the resulting state.

14. Learning Scientist and asynchronous session analysis

The Learning Scientist subsystem analyzes evidence after a meaningful learning session has ended or timed out. It must not block the normal learning interaction unnecessarily.

14.1 Responsibilities

It may summarize validated session evidence, propose learner-state adjustments, identify unresolved diagnostic questions, review intervention outcomes, propose retrieval schedule updates, and produce auditable analysis results.

14.2 Processing rules

Session analysis must:

- Use a defined, validated slice of the event ledger.
- Be idempotent for the same session and analysis version.
- Record the input event range and policy and model versions.
- Avoid double-applying an already committed state update.
- Preserve the distinction between observed evidence and inferred conclusions.
- Recover from worker or database failures.
- Avoid training shared-model weights as a side effect of ordinary session processing.

14.3 Approval boundary

Decision required (D-11): define which outputs the Learning Scientist may commit automatically and which require validation by a deterministic policy or another approved mechanism. Until that contract is approved, the subsystem must not silently overwrite authoritative learner state.

15. Answer verification and evidence provenance

LAM distinguishes a reported answer outcome from a verified answer outcome. Supported provenance categories include:

- Symbolic mathematics check
- Code tests
- Answer key
- Curriculum fact check
- Teacher verification
- Model-graded result

The final enum, semantics and trust policy are specified in the event contract (decision D-12). A model-graded result remains provisional where the approved policy requires independent verification, and must not alone establish mastery if that would violate the mastery policy.

The host application is responsible for performing or obtaining answer verification and returning its provenance to LAM. LAM validates that the supplied provenance is permitted for the event type and caller.

16. Shared Learning Adaptation Model

The shared model learns general patterns across approved data. It does not replace a student's private learner state.

16.1 Separation from live processing

Shared-model training stays outside the live event-ingestion and recommendation path. A new learning event must not automatically update shared-model weights.

16.2 Required lifecycle

1. Data eligibility rules
2. Approval and preparation of a training dataset
3. Privacy and governance checks
4. A versioned training procedure
5. A fixed evaluation dataset and baseline
6. Evaluation of learning outcomes and failure cases

7. Model registration and versioning
8. Staged rollout
9. Monitoring and rollback
10. A documented record of the approved model and evaluation

16.3 Data governance

Consent, retention, deletion, correction and training eligibility must be defined before real student data is used for shared training (decision D-13). Identifiable student histories must not be stored in the shared model, and raw conversations are not used for training by default. The training pipeline enforces eligibility itself rather than relying only on a client-side flag.

16.4 Promotion requirements

A new model or policy is not promoted merely because training completes successfully. It is compared with the existing baseline through a documented evaluation covering independent mastery, delayed retention, transfer, uncertainty and relevant failure cases (decision D-16). M4 must not begin until the deterministic baseline and evaluation process are established well enough to support a meaningful comparison.

17. Decision register: mandatory before affected implementation

These decisions must be resolved and recorded in `docs/decisions/` before engineers implement the corresponding complex behavior.

ID	Decision	Required resolution
D-01	Mastery policy	Approve the estimate threshold, evidence-count requirements, session separation, difficulty requirements and delayed-retention requirements
D-02	BKT configuration	Approve the model variant, parameters, initialization and parameter-versioning strategy
D-03	Uncertainty semantics	Define what uncertainty represents, how it is calculated and how it affects progression

ID	Decision	Required resolution
D-04	Diagnosis confirmation	Approve confidence semantics, evidence-independence rules and hypothesis lifecycle thresholds
D-05	Assistance ladder	Approve level mappings, escalation transitions, reset rules and independence requirements
D-06	Decision priority	Approve hard constraints, priority ordering, tie-breaking and fallback behavior
D-07	Retrieval scheduler	Approve baseline interval, multipliers, caps, overdue handling and failure resets
D-08	Event idempotency	Approve duplicate-event responses and conflicting reuse of event identifiers
D-09	Identity and tenancy	Approve trusted identity propagation, service authentication and tenant authorization
D-10	Curriculum adapter	Approve graph schema, version compatibility and behavior when the graph is unavailable
D-11	Learning Scientist	Approve asynchronous processing, retry, commit and state-recalculation semantics
D-12	Verification provenance	Approve supported provenance types and which can contribute to mastery confirmation
D-13	Privacy lifecycle	Approve consent, retention, deletion, correction and training eligibility
D-14	Operational targets	Approve latency, throughput, availability, recovery and observability targets
D-15	API and SDK compatibility	Approve versioning, deprecation, error contracts and supported client compatibility
D-16	Evaluation thresholds	Approve baseline datasets, outcome metrics, minimum evidence and model/policy promotion gates

Each decision record contains: the problem, options considered, selected resolution, rationale, affected modules, migration or compatibility impact, tests, owner and approval date.

Until a decision is approved, engineers may implement unaffected foundations and interfaces, but must not invent an authoritative policy for the unresolved behavior.

18. Testing and acceptance requirements

18.1 Unit tests

Unit tests cover domain rules and deterministic functions without requiring production infrastructure. Required areas:

- Event validation, and missing and unknown values
- Mastery updates, and independent versus assisted evidence
- State initialization
- Diagnosis candidate generation and hypothesis lifecycle transitions
- Assistance-level transitions
- Prerequisite eligibility
- Decision priority and tie-breaking
- Retrieval interval changes
- Reason-code generation
- Version compatibility
- Failure and fallback behavior

18.2 Integration tests

Integration tests use a real isolated PostgreSQL test database or an equivalent faithful environment. They verify:

- Migrations and schema constraints
- Event insertion and persistence
- Duplicate event behavior
- Transaction rollback after partial failure
- Concurrent learner-state updates
- State retrieval after process restart
- Tenant isolation
- Recovery after temporary database failures
- Audit records and correction workflows

Mocks alone are not sufficient proof of database correctness.

18.3 Contract tests

- Request and response schemas
- Stable error codes
- Authentication and authorization
- Pagination and response limits
- Idempotent event submission
- State-version conflict behavior
- Python SDK and TypeScript client compatibility
- Qrie integration behavior
- API-version compatibility

18.4 Security and privacy tests

Tests must prove that:

- Student A cannot access or modify Student B's records.
- Tenant A cannot access Tenant B's records.
- Missing, invalid or expired credentials are rejected.
- Administrative and background paths enforce authorization.
- Sensitive content and credentials do not leak into logs.
- Data retention and deletion behavior follows the approved policy.
- Untrusted inputs and model outputs are validated before use.

18.5 End-to-end learning test

1. An authorized learner submits a learning event.
2. LAM validates and stores the evidence.
3. Learner state is initialized or updated.
4. A recommendation is requested.
5. LAM returns a policy-compliant recommendation.
6. The host application delivers the selected intervention.
7. The learner's resulting evidence is submitted.
8. LAM distinguishes assisted and independent performance.

9. A review is scheduled when required.
10. A delayed assessment is submitted.
11. The resulting state and review plan reflect the validated evidence.
12. The full process can be audited and reproduced according to the documented versioning rules.

18.6 Required manual scenarios before M2

- First event from a fresh learner
- Correct independent answer
- Correct answer after a hint
- Wrong answer with an ambiguous cause
- Repeated failure and assistance escalation
- Blocking prerequisite gap
- Duplicate event submission
- Concurrent state updates
- Missing or invalid curriculum data
- Unavailable database or downstream dependency
- Cross-student and cross-tenant access attempts
- Delayed retrieval success and failure

Each manual record contains: test ID, requirement or issue, commit SHA, build, environment, database migration version, policy and model version, preconditions, exact steps, expected and actual results, pass/fail/blocked status, evidence, tester, date, defects and retest result.

19. CI/CD and engineering workflow

Every change follows the mandatory engineering lifecycle:

1. Create or update an issue with problem context, scope, acceptance criteria, privacy and security impact, and test plan.
2. Write or update the design decision or contract for behavior-changing work.
3. Create a small branch from the approved base.
4. Implement one testable change at a time.
5. Add tests alongside the implementation.

6. Run formatting, linting, type checking, unit tests, relevant integration tests and applicable security checks.
7. Perform required manual verification.
8. Open a pull request with risks, tests, evidence and rollback considerations.
9. Obtain independent review for behavior-changing work.
10. Merge only when required checks pass and acceptance criteria are satisfied.
11. Deploy to a non-production environment first.
12. Run smoke and regression checks.
13. Verify the deployed version, logs, metrics and a representative learning flow.
14. Roll back when a documented stop condition is met.

19.1 Mandatory quality gates

At minimum, CI runs:

```
ruff check .
ruff format --check .
mypy --strict src/lam
pytest
```

The exact commands and configuration may be adjusted to the final project structure, but required gates must be enforced by the repository rather than left as optional local recommendations. Integration, contract and security test jobs are added as the relevant infrastructure becomes available. No engineer may bypass a required gate merely to merge a change.

20. Observability, performance and recovery

20.1 Observability

The service provides structured, privacy-safe observability for:

- Request and correlation IDs
- Event processing status
- Processing and recommendation latency
- Error categories and rates
- Duplicate-event counts and state-version conflicts

- Database availability and failed asynchronous jobs
- Policy, schema, graph and model versions
- Recommendation reason codes
- Retrieval-scheduler outcomes

Do not log raw student transcripts, answer payloads, access tokens, secrets or sensitive content unless an explicitly approved, minimized use case requires it.

20.2 Performance requirements

The source documents establish the need for reliable, independently deployable service behavior but give no final numerical performance targets.

Decision required (D-14): set target latency for event ingestion and recommendation requests, expected concurrent learners, throughput, availability, maximum payload size, timeouts, retry limits and recovery objectives, based on actual classroom requirements. Until these are approved, performance claims must not be presented as verified service guarantees.

20.3 Failure behavior

The system distinguishes between invalid requests (rejected), unauthorized requests (denied), recoverable infrastructure failures, insufficient evidence for a learning inference, invalid or unavailable curriculum information, conflicting state updates, and asynchronous work that has not yet completed. Each case has a documented response, retry policy, audit behavior and safe fallback where appropriate.

21. Milestones and release gates

M0: Service foundation

Scope: repository scaffold; typed domain and API contracts; authentication and tenant authorization; PostgreSQL setup and migrations; versioned API; durable, idempotent event ledger; initial learner-state storage contract; CI and quality gates; SDK skeleton; basic observability; unit, integration, contract and security test foundations.

Acceptance: valid events can be ingested and retrieved according to the approved contract; duplicate events do not double-apply logical changes; tenant isolation and database integrity are tested; migrations are reproducible; CI is enforced; and the service can be deployed and health-checked in a non-production environment.

M1: Deterministic learner model

Scope: per-concept learner state; approved BKT baseline; assisted and independent evidence separation; initial diagnosis hypotheses; rule-based next-action selection; state versioning and auditability; deterministic unit and integration tests.

Acceptance: the approved baseline produces reproducible state updates and recommendations for the fixed test scenarios. Insufficient evidence remains insufficient; assisted success does not establish independent mastery; and policy decisions can be explained using recorded evidence and version identifiers.

M2: Closed-loop Qrie integration

Scope: Qrie integration adapter; curriculum graph contract; diagnostic interventions; assistance policy; independent assessments; retrieval scheduling; delayed assessments; persistent state updates across sessions.

Acceptance: the full learning loop works through the host application, including evidence submission, recommendation delivery, subsequent outcome ingestion, delayed retrieval and auditable state updates.

M3: Validated personalization

Scope: strategy-effectiveness tracking; uncertainty calibration; improved recommendation policy; fixed benchmark scenarios; baseline comparison and policy evaluation.

Acceptance: personalization improvements are supported by evaluation results. Changes do not regress prerequisite constraints, student isolation, independent mastery verification or safety behavior.

M4: Governed shared intelligence

Scope: approved training data; offline training and evaluation pipeline; versioned model artifacts; model registry or equivalent artifact management; staged rollout; monitoring and rollback.

Acceptance: a candidate model is evaluated against the approved baseline, satisfies the promotion criteria, has documented provenance, and can be rolled back. No live event automatically trains shared-model weights.

21.1 Global release gate

A milestone must not be marked complete solely because its endpoints work. Before release, verify:

- Acceptance criteria are met.
- Required CI checks pass.
- No unresolved P0 or P1 defects remain.
- Relevant security and privacy tests pass.
- Required manual evidence is recorded.
- Database migrations and rollback considerations are reviewed.
- Policy and schema versions are documented.
- The deployed learning flow is verified in the target environment.

22. Definition of Done

A feature is complete only when:

1. Its requirement and issue are linked.
2. Its design and relevant decisions are documented.
3. Inputs, outputs and failure behavior are specified.
4. Implementation follows the dependency rules.
5. Tests cover normal, boundary, missing, invalid, concurrent and failure cases where relevant.
6. Required integration and security tests pass.
7. Manual verification is recorded where required.
8. State-changing behavior is auditable and versioned.
9. Authorization and tenant isolation are verified.
10. Documentation reflects the actual implementation.
11. CI gates pass.
12. Risks and rollback considerations are recorded.
13. No unsupported claim of mastery, diagnosis or model performance is made.
14. The feature satisfies its milestone acceptance criteria.

A successful HTTP response is not proof of a completed feature.

23. Final implementation directive

Build LAM incrementally, beginning with M0 and the smallest testable vertical slice.

Do not begin with sophisticated machine learning, multi-agent orchestration or a collection of microservices. Establish durable evidence, authorization, deterministic state transitions, reproducible policies and a tested integration contract first.

Before implementing complex learning logic, approve the decision register and write executable algorithm specifications for mastery estimation, diagnosis, assistance escalation, recommendation selection and retrieval scheduling.

The intended outcome is an independently deployable, auditable learning-adaptation service that helps learners become more capable of learning independently over time, not merely a recommendation API or a wrapper around a language model.

How to use this document

Save the draft as `docs/LAM_IMPLEMENTATION_SPECIFICATION.md`. Keep the existing requirement and standards documents as the original sources; do not replace them with this one.

Before implementing the complex learning logic, complete these three actions:

1. Review and approve section 17 with the technical owner. The mastery, diagnosis and retrieval policies need one authoritative version.
2. Create `docs/algorithm-specifications.md` with exact pseudocode, inputs, outputs, edge cases and test vectors for the deterministic algorithms.
3. Create requirement-to-test traceability, mapping each functional requirement to its module, API contract, automated tests, manual verification and milestone.

Qualification: this is a detailed implementation-specification draft derived from the source documents and the previously established LAM architecture. It is not yet an approved specification, and it does not establish that the current repository or any implementation meets these requirements.